IP HARDENING OF PRESENT CIPHER ENCRYPTION SBOX MODULE

Technology Node:

SCL 180nm CMOS PDK

Design Flow:

RTL Linting | Logic Synthesis | Physical Design | Signoff

Date:

June 23, 2026

Tool Suite:

JasperGold | Genus | Innovus | Tempus (Cadence)

Submitted by :

G R Shivaranjani

23BEC1121,

SENSE, VIT CHENNAI

Submitted to :

Dr.A.Prathiba

Associate Professor,

SENSE, VIT CHENNAI

TABLE OF CONTENTS

1. About the Design

The PRESENT cipher is a lightweight block cipher designed for resource-constrained environments such as RFID tags, sensor nodes, and embedded security modules. It operates on 64-bit data blocks using either an 80-bit or 128-bit key, making it highly suitable for hardware implementations where silicon area, power, and throughput are all critical.

The SBOX (Substitution Box) is the sole non-linear component of the PRESENT cipher, providing the cryptographic confusion property essential to its security. This report documents the complete IP hardening flow of the SBOX module, starting from RTL verification in JasperGold through logic synthesis in Cadence Genus, and culminating in full physical implementation and signoff using the SCL 180nm CMOS PDK in Cadence Innovus.

1.1 Design Hierarchy

The top-level module SBoxIntegration comprises nine sub-module instances across seven unique module types:

- I1 (isomorphic) — Isomorphic mapping from GF(2^8) to composite field GF(((2^2)^2)^2)
- Square (SquareOrInv) — Squaring/inversion shared logic in GF(2^4)
- P1 (PhaseBlock) — Phase computation block
- M1, M2, M3 (Multiplier) — Composite field multipliers
- Inverse (SquareOrInv) — Field inversion
- IV1 (InverseIsomorphic) — Inverse isomorphic mapping back to GF(2^8)
- A1 (AffineTransform) — Affine transformation producing the final SBOX output

1.2 RTL File Set

- SBoxIntegration.v (741 B) — Top-level integration module
- PhaseBlock.v (621 B) — Phase block
- SquareOrInv.v (623 B) — Square/Inversion shared logic
- AffineTransform.v (382 B) — Affine transformation
- isomorphic.v (227 B) — Isomorphic mapping
- InverseIsomorphic.v (208 B) — Inverse isomorphic mapping
- Multiplier.v (194 B) — Composite field multiplier

2. RTL Linting

RTL linting was performed using Cadence JasperGold's Superlint application. Formal-verification-based lint checking ensures that the RTL is synthesizable, structurally clean, and free of common design hazards before proceeding to synthesis.

2.1 Superlint Initialization and Verilog Analysis

The Superlint application was initialized and all seven Verilog source files were analyzed:

```
% check_superlint -init
% analyze -verilog {/home/userdata/22bec0985/sbox/SBoxIntegration}
```

Figure 2.1: Superlint initialization and Verilog analysis in JasperGold

2.2 Elaboration

The design was elaborated with SBoxIntegration as the top module using:

```
% elaborate -top SBoxIntegration
```

The elaboration summary reported 7 synthesized netlists across 7 pipelined netlists, with all 7 Verilog modules successfully analyzed. Late Hierarchical Reference Connection was enabled and the state was cleared for a clean analysis environment.

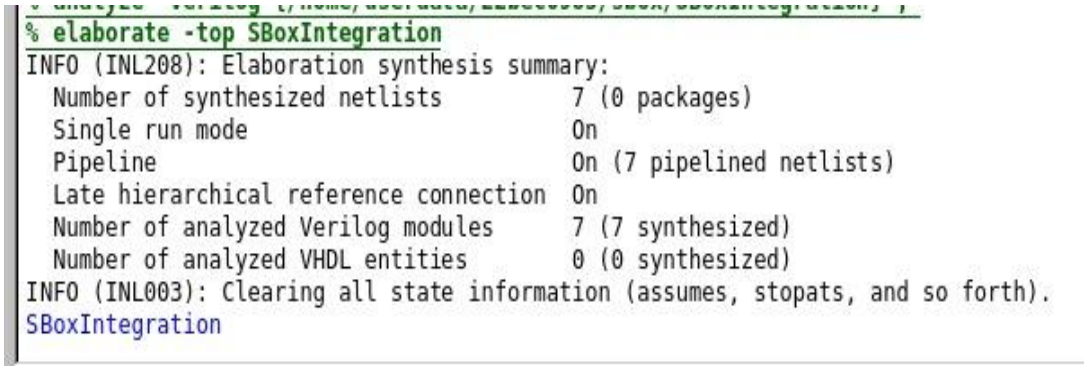


Figure 2.2: Elaboration summary — 7 synthesized netlists confirmed

2.3 Clock Declaration

A clock was declared to enable sequential property checking. Settings used: Clock name: clk, Reference clock: clk, Factor: 1, Phase: 1. The both-edges trigger was left unchecked (positive-edge clocking only).

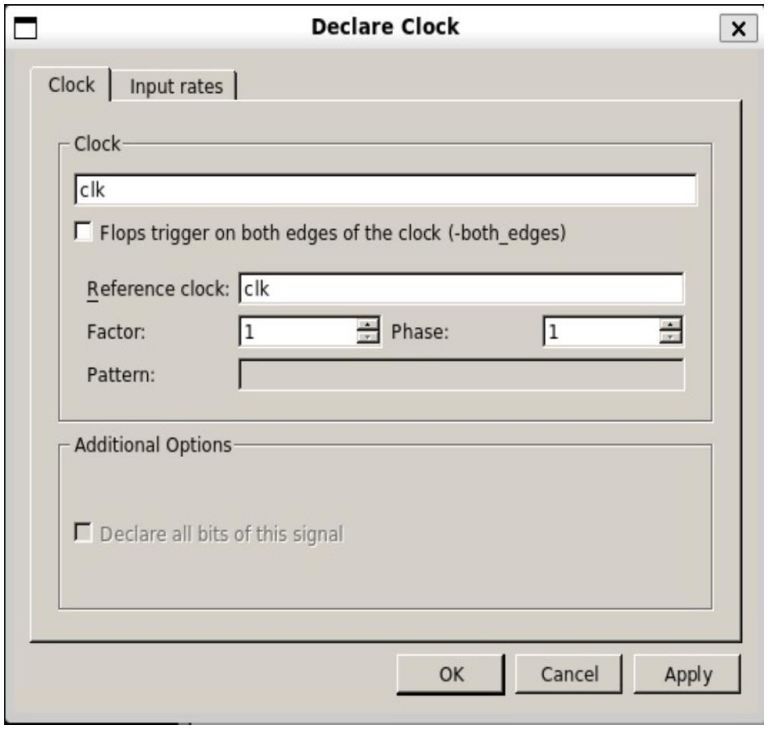


Figure 2.3: Declare Clock dialog in JasperGold Superlint

2.4 Generate and Prove

The Generate and Prove function was run across the full instance hierarchy. All 9 instances of SBoxIntegration were selected. The 'Modify instances to extract' option was enabled for fine-grained extraction control.

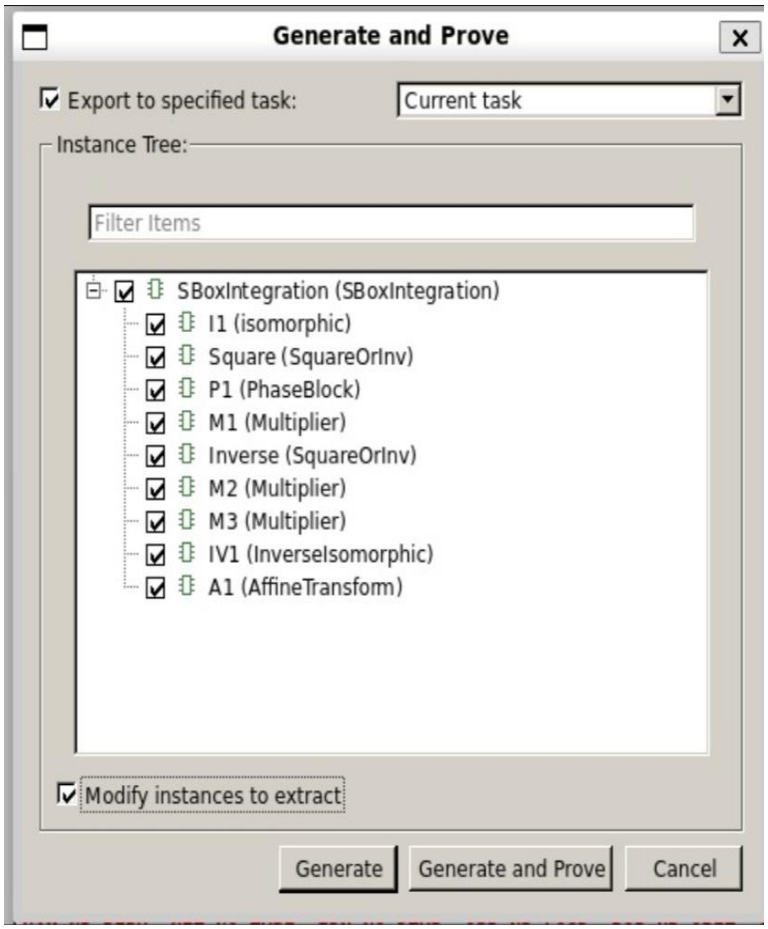


Figure 2.4: Generate and Prove dialog — complete SBoxIntegration hierarchy selected

2.5 Lint Violation Summary

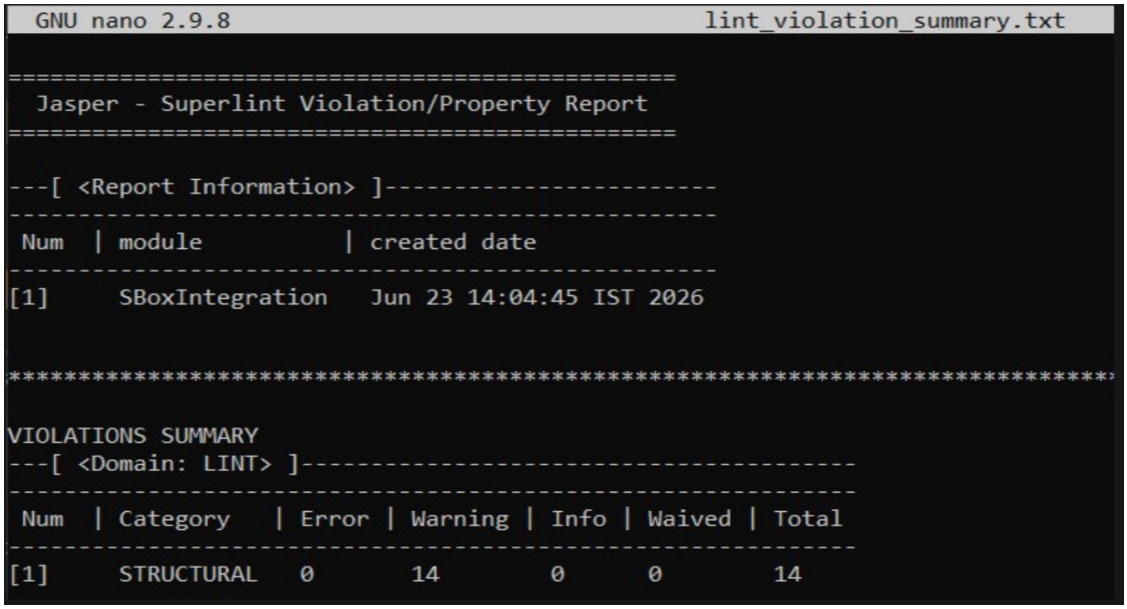


Figure 2.5: Jasper Superlint Violation/Property Report — SBoxIntegration (Jun 23, 2026)

Category	Error	Warning	Info	Total
STRUCTURAL	0	14	0	14

The lint run produced 14 structural warnings and 0 errors. Zero errors confirm the RTL is structurally legal and synthesizable. The 14 warnings are informational and do not affect functional correctness or synthesis quality for this lightweight cipher implementation.

3. Synthesis

Logic synthesis was performed using Cadence Genus targeting the SCL 180nm standard cell library. The synthesized gate-level netlist includes IO pad instantiations for all primary ports, making the design ready for physical implementation with a complete IO ring.

3.1 Technology Library — SCL 180nm

- Process node: 180nm CMOS (Semiconductor Laboratory, India)
- Supply voltage: 1.8V nominal core, 3.3V IO
- Standard cell library with multiple drive-strength variants
- IO pad library: tsl18cio150_4lm — 4-metal-layer IO cells with ESD protection
- Timing corners: SS (slow-slow) at 1.62V/125°C and FF (fast-fast) at 1.98V/−40°C

3.2 IO Pad Instantiation

All primary ports were appended with the suffix '_pad' and mapped to SCL 180nm IO pad cells. The pad types used include input data pads, output data pads, a buffered clock input pad, and corner cells. IO filler pads (pfeed family) fill the remaining IO ring periphery.

3.3 Timing Constraints

An SDC file (timing.g) was applied specifying: clock definition on clk at the target operating frequency, input/output delay constraints relative to the clock, and false path exclusions for scan logic.

3.4 Synthesis Results

All 7 RTL modules were successfully mapped to SCL 180nm cells. The synthesized netlist (SBOX_incremental.v) was verified for functional equivalence using Cadence Conformal LEC and passed. The netlist was then imported into Cadence Innovus for physical implementation.

4. Physical Design

Physical design was performed in Cadence Innovus Implementation System 21.15. The flow covers MMMC setup, design import, floorplanning, power planning, placement, CTS, routing, and signoff. All steps used the SCL 180nm LEF/Liberty libraries and a full IO pad ring methodology.

4.1 Design Import and MMMC Setup

4.1.1 RC Corners

Two RC corners were defined to bound parasitic extraction across process, voltage, and temperature (PVT) extremes:

- rc_worst: Temperature 125°C — models worst-case resistance/capacitance for setup timing
- rc_best: Temperature −40°C — models best-case parasitics for hold timing

Both corners used the cap table file _26092019_basic.CapTbl with PostRoute resistance and cap scale factors of 1.0.

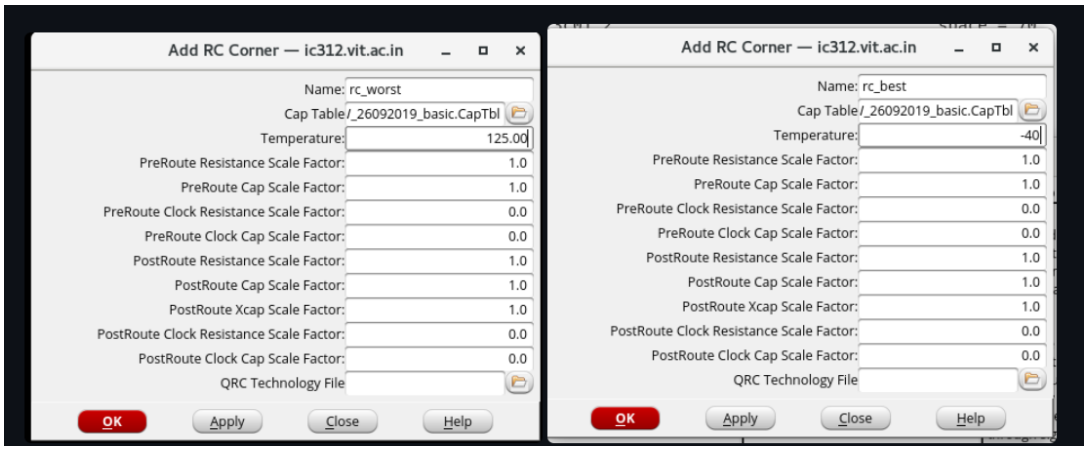


Figure 4.1: RC Corner definitions — rc_worst (125°C) and rc_best (−40°C)

4.1.2 Operating Conditions

Two operating conditions were defined corresponding to the SS and FF library corners:

- op_cond_worst: Library tsl18fs120_scl_ss.lib, Process 1.0, Voltage 1.62V, Temperature 125°C
- op_cond_best: Library tsl18fs120_scl_ff.lib, Process 1.0, Voltage 1.98V, Temperature −40°C

Figure 4.2: Operating Condition definitions — worst (SS corner) and best (FF corner)

4.1.3 Delay Corners and MMMC View

A max_delay corner was configured using On-Chip Variation (OCV) mode. The Early timing arc uses the FF library (min_timing / op_cond_best) and the Late arc uses the SS library (max_timing / op_cond_worst), with rc_worst parasitics. This is the standard OCV setup for robust STA.

Figure 4.3: Edit Delay Corner — max_delay with OCV, Early=FF/best, Late=SS/worst

The MMMC Browser confirmed the complete multi-mode multi-corner configuration with: worst/best Analysis Views, Setup Analysis View (worst), Hold Analysis View (best), two RC corners, two OP conditions, two delay corners (max_delay, min_delay), and a single constraint mode (all).

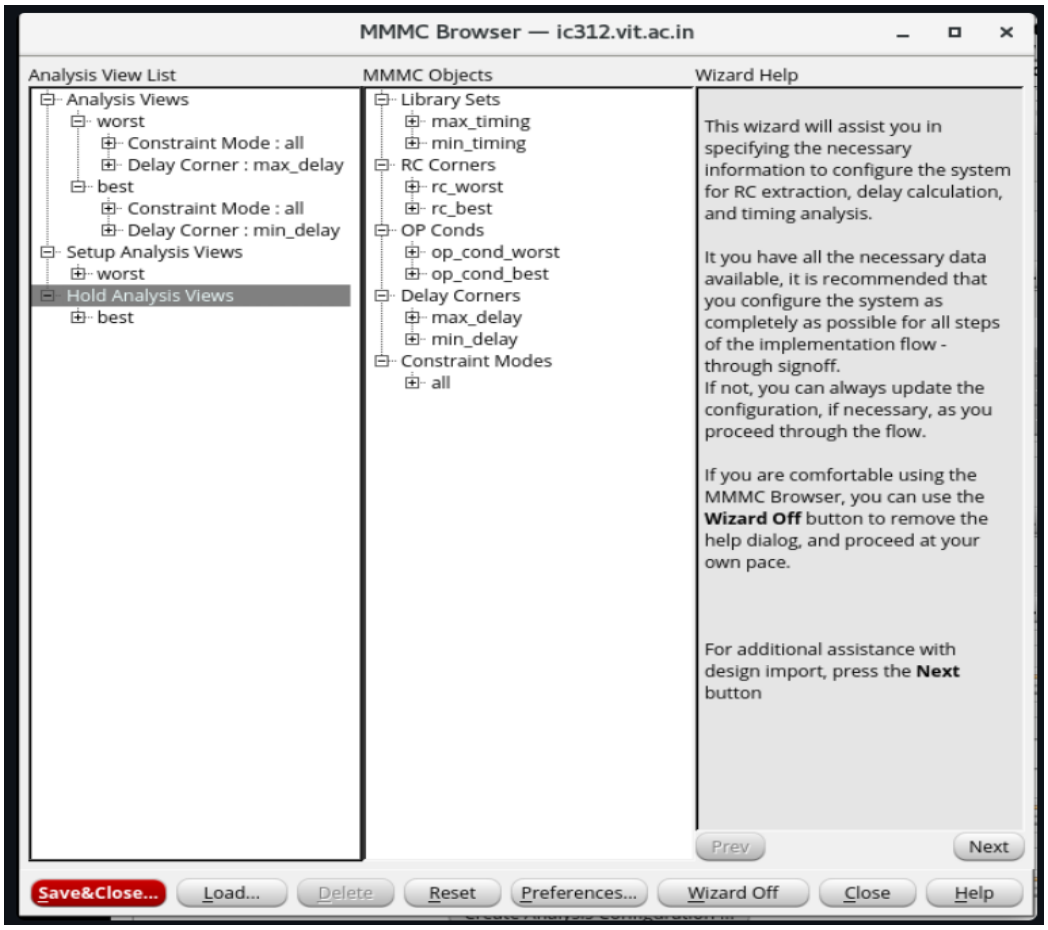


Figure 4.4: MMMC Browser — complete timing configuration for worst/best analysis views

4.1.4 Design Import

The synthesized gate-level netlist was imported into Innovus with the following configuration:

- Netlist: ../output_files/SBOX_incremental.v (Verilog), Top Cell: SBoxIntegration
- LEF Files: scl18fs120_tech.lef, scl18fs120_std.lef, tsl18cio150_4lm.lef
- IO Assignment File: pins.io
- Power Nets: VDD_CORE, VDDO_CORE; Ground Nets: VSS_CORE, VSSO_CORE
- MMMC View Definition File: sbox.view

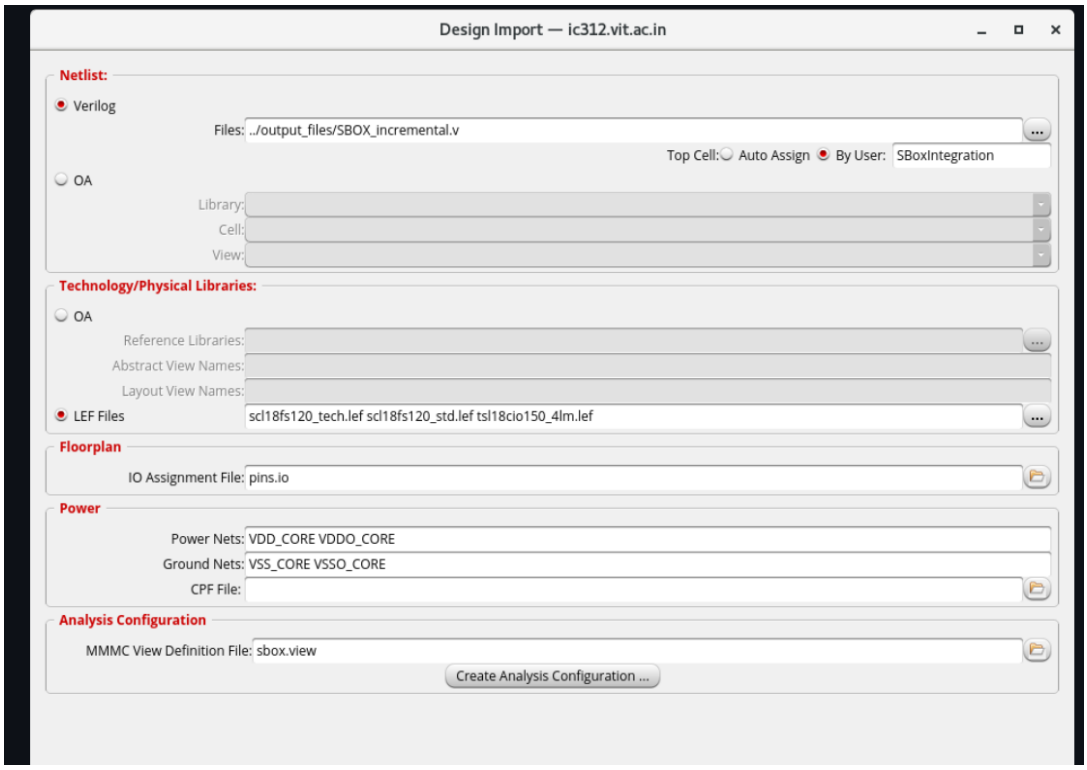


Figure 4.5: Design Import dialog — netlist, LEF libraries, IO assignment, power nets and MMMC view

After import, the initial Innovus layout showed IO pad instances that were not yet constrained to the peripheral boundary — a situation that was corrected during the floorplanning step.

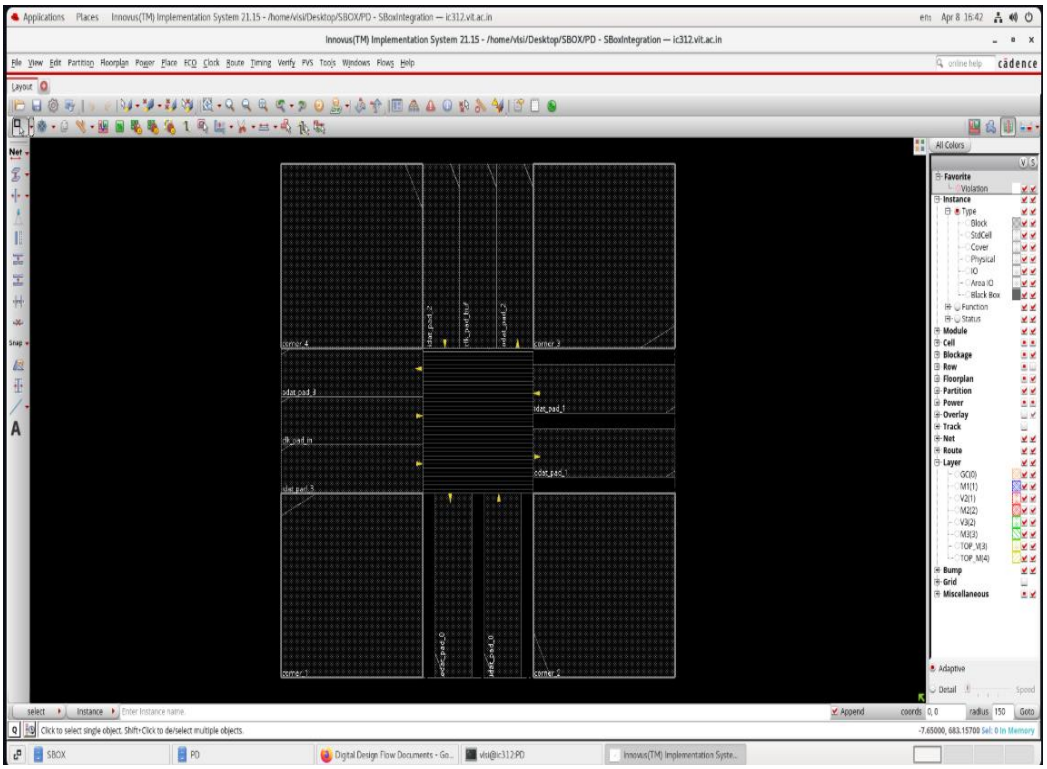


Figure 4.6: Initial Innovus layout post design import (IO pads not yet placed in peripheral ring)

4.2 Floorplanning

The floorplan was specified using the Specify Floorplan dialog with the following dimensions:

- Die size: 1939.84 × 1939.84 µm (approximately 1.94 mm × 1.94 mm)
- Core size: 1190.08 × 1190.08 µm
- Core-to-IO boundary margins: 124.88 µm on all four sides
- IO Box Calculation: Min IO Height; Floorplan Origin: Lower Left Corner

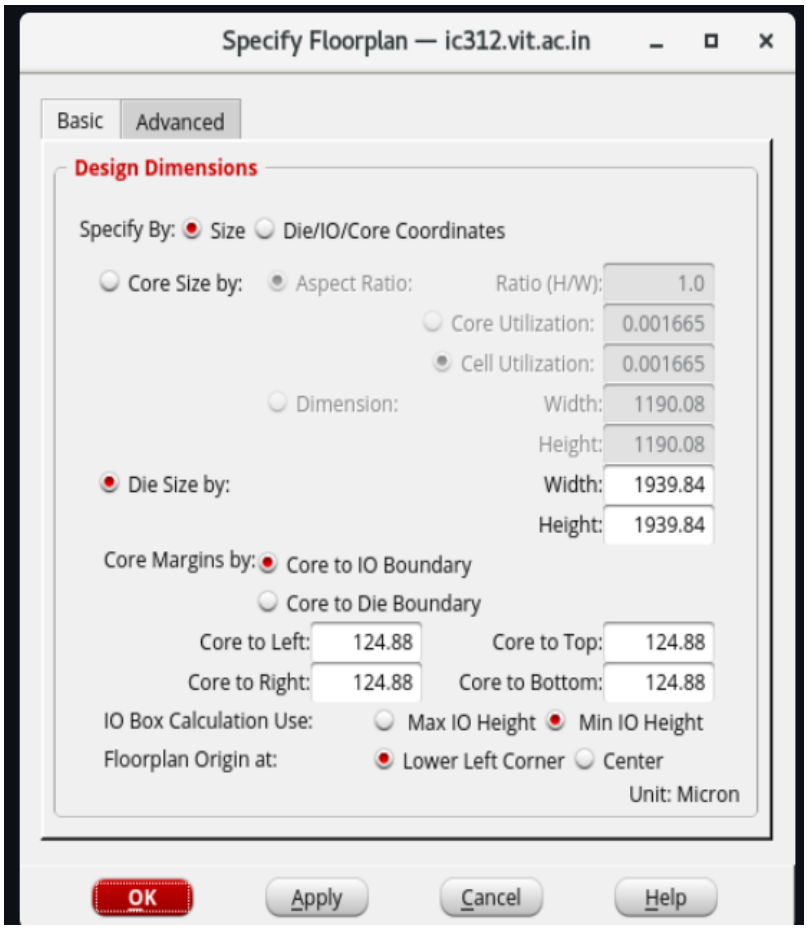


Figure 4.7: Specify Floorplan dialog — 1939.84×1939.84 µm die, 124.88 µm margins

The IO pads were placed into the peripheral ring using a custom pins.io assignment file. This correctly mapped all IO pad instances (idat_pad, odat_pad, clk_pad, corner cells) to the four sides of the chip boundary.

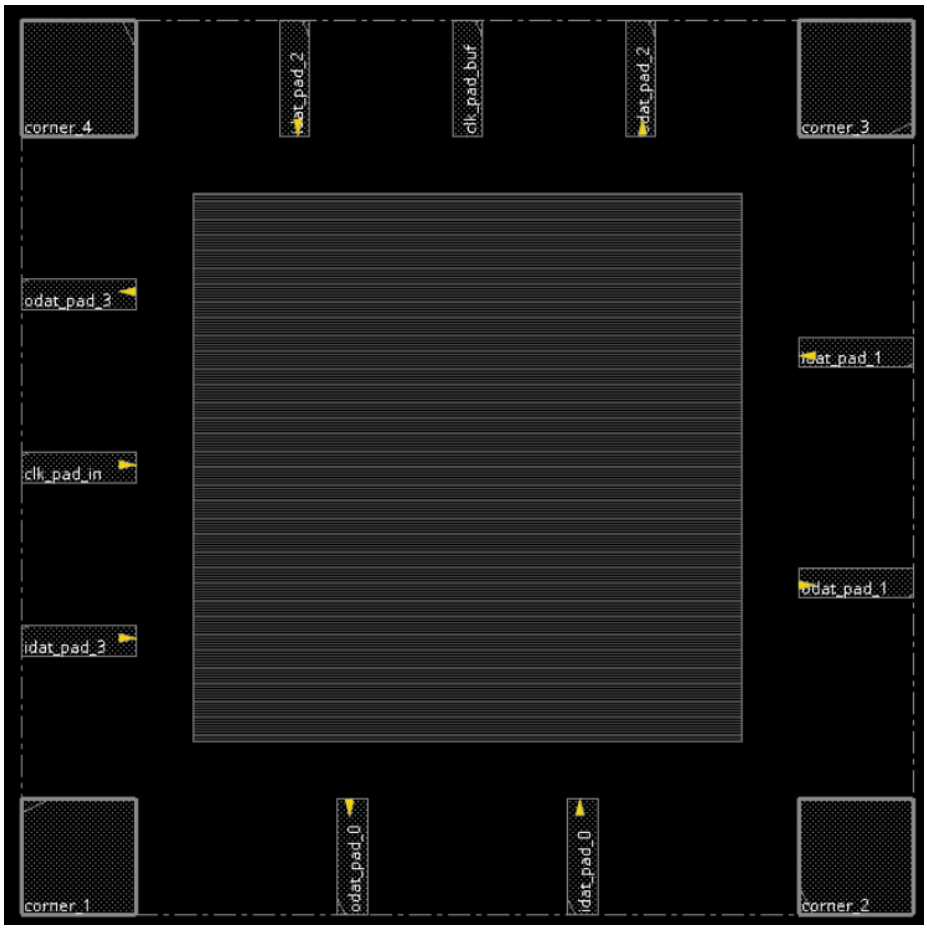


Figure 4.8: Floorplan with IO pads placed in peripheral ring — before IO filler insertion

4.3 IO Filler Insertion

IO filler cells (pfeed family) were inserted to fill all remaining gaps in the IO pad ring, ensuring well continuity around the full chip periphery. The insertion flow:

- Step 1: Delete any pre-existing IO fillers — deleteInst *IOFILLER*
- Step 2: Re-add with the correct cell list:

```
addIoFiller -cell {pfeed30000 pfeed10000 pfeed02000 pfeed01000 pfeed00540 pfeed00120} -prefix IOFILLER
```

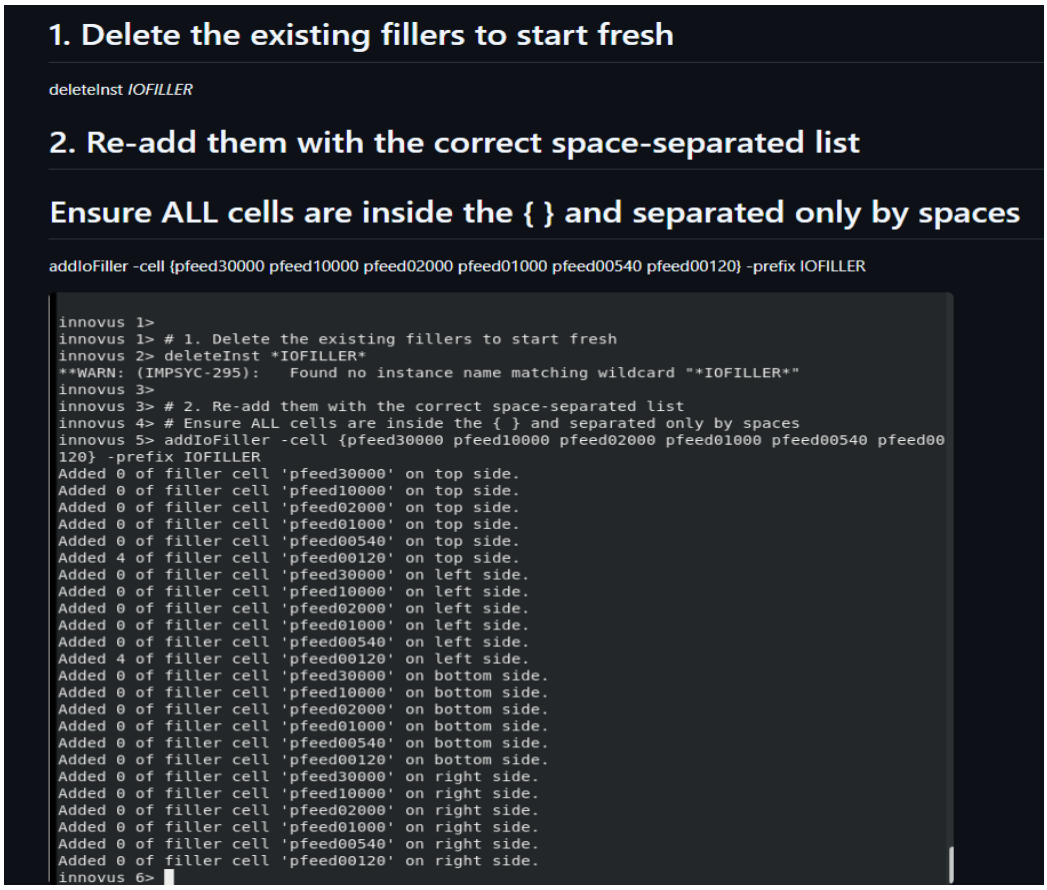


Figure 4.9: IO filler insertion console — pfeed cells added on all four sides

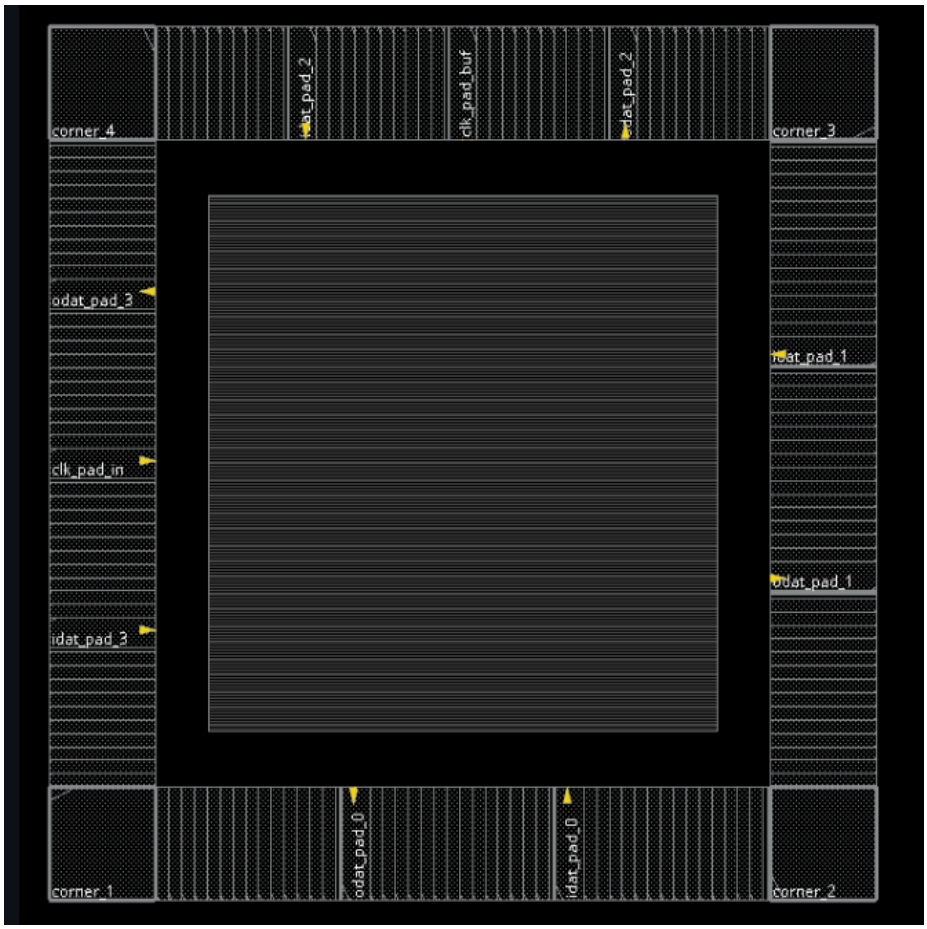


Figure 4.10: Post-IO-filler floorplan — complete peripheral IO pad ring with all gaps filled

4.4 Power Planning

4.4.1 Global Net Connections

Power and ground nets were connected globally using the following Innovus commands:

```
globalNetConnect VDD_CORE -type pgpin -pin VDD -all
globalNetConnect VSS_CORE -type pgpin -pin VSS -all
globalNetConnect VDDO_CORE -type pgpin -pin VDDO -all
globalNetConnect VSSO_CORE -type pgpin -pin VSSO -all
globalNetConnect VDD_CORE -type tiehi
globalNetConnect VSS_CORE -type tielo
```

4.4.2 Power Rings

Core power rings were added around the core boundary on metal layers M3 (horizontal) and TOP_M4 (vertical), with the following configuration:

- Nets: VDD_CORE, VSS_CORE
- Ring type: Core ring contouring around core boundary
- Top/Bottom layers: M3(3) H, width 25 µm, spacing 10 µm, offset 1.8 µm
- Left/Right layers: TOP_M4 V, width 25 µm, spacing 10 µm, offset 1.8 µm

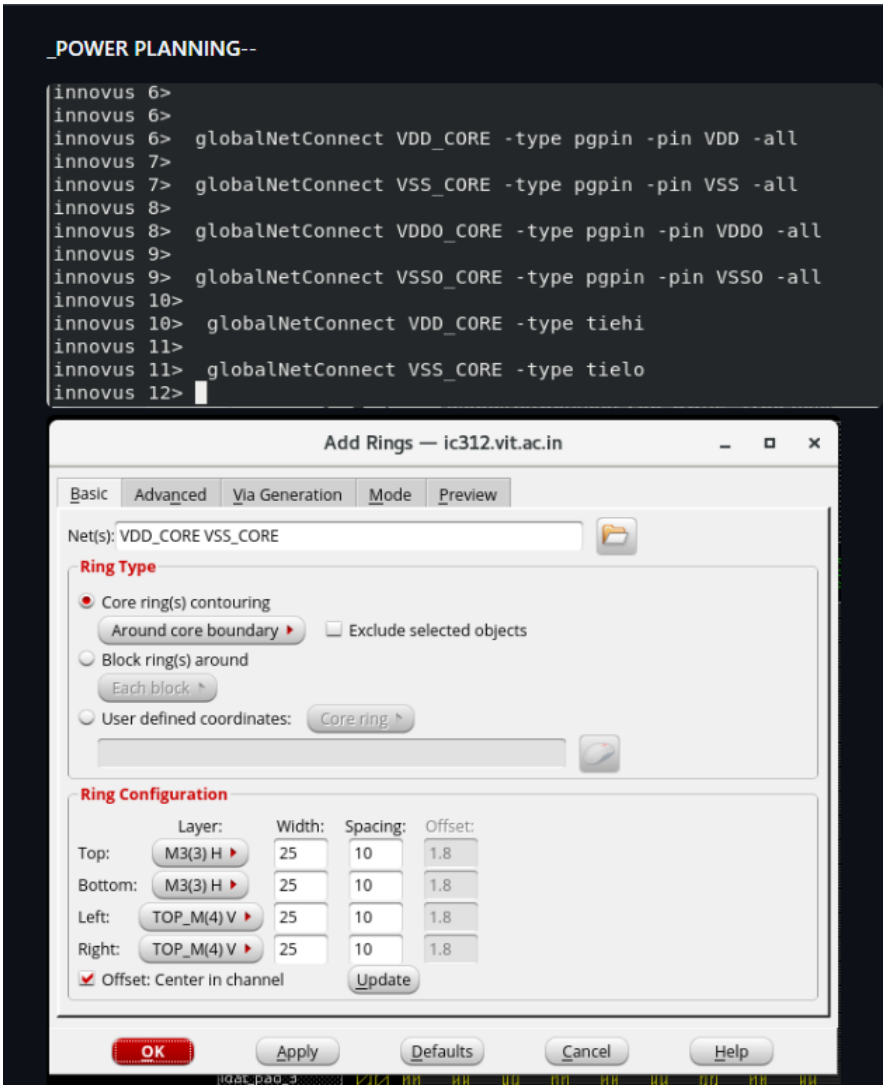


Figure 4.11: Power Planning — global net connections and Add Rings configuration (M3/TOP_M4, width 25)

4.4.3 Power Stripes

Vertical power stripes were added on TOP_M4 to create a robust power mesh across the core:

- Nets: VDD_CORE, VSS_CORE
- Layer: TOP_M4, Vertical direction
- Width: 10 µm, Spacing: 10 µm, Set-to-set distance: 100 µm
- Stripe boundary: Core ring

Figure 4.12: Add Stripes dialog — VDD/VSS vertical stripes on TOP_M4, width 10, spacing 10, pitch 100 µm

4.4.4 SRoute — Power Rail Connection

The SRoute command was used to connect the power rings and stripes to the standard cell power/ground rails, completing the power delivery network:

- Nets: VDD_CORE VSS_CORE
- Layer range: TOP_M4 (top) to M1 (bottom)
- Options: Floating Stripes, Pad Pins, Follow Pins enabled; Allow Jogging and Layer Change enabled

Figure 4.13: SRoute dialog — power rail connection from TOP_M4 to M1 with pad pin connections

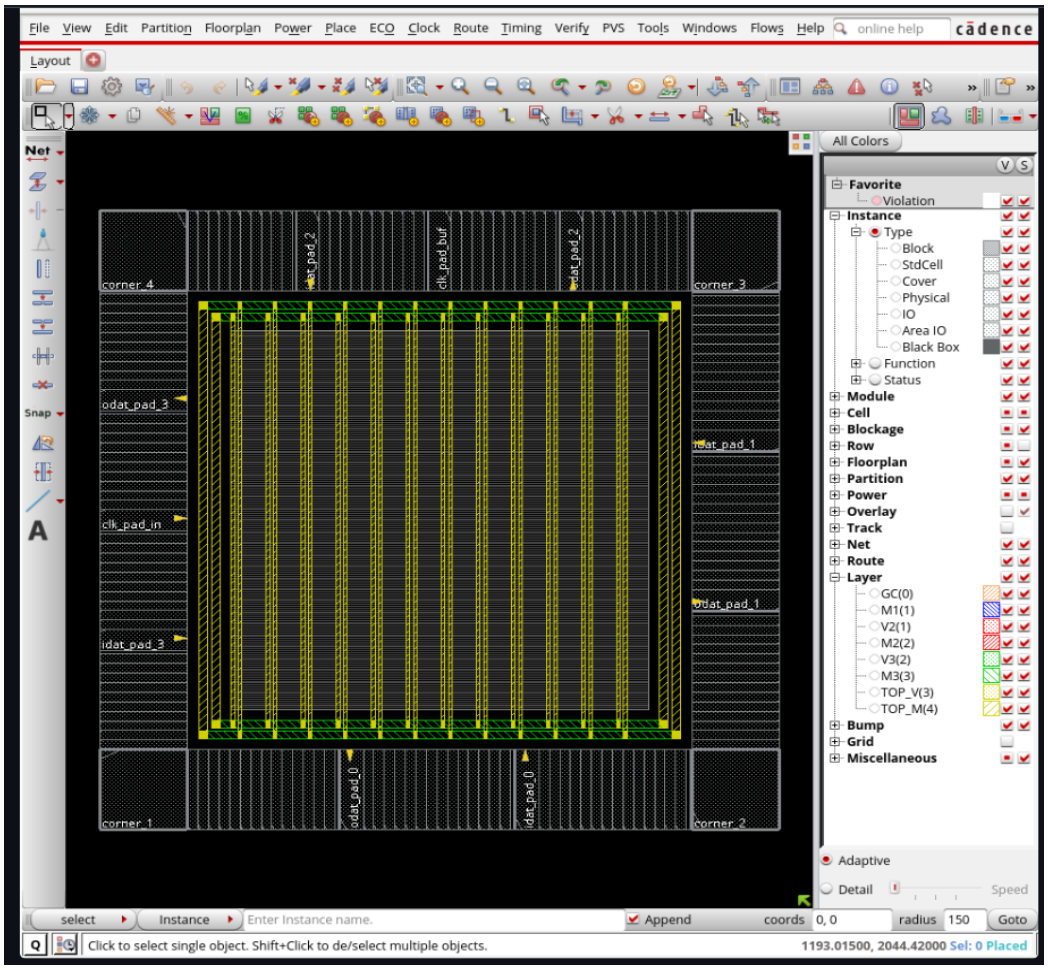


Figure 4.14: Post power planning layout — VDD/VSS power grid (yellow/green stripes) visible across core

4.5 Placement

Standard cell placement was run with Congestion Effort set to HIGH to proactively spread cells and prevent routing hotspots. Additional placement settings:

- Run Timing Driven Placement: enabled
- Enable Clock Gating Awareness: enabled
- Ignore Scan Connections and Reorder Scan Connection: enabled
- Specify Maximum Routing Layer: 1

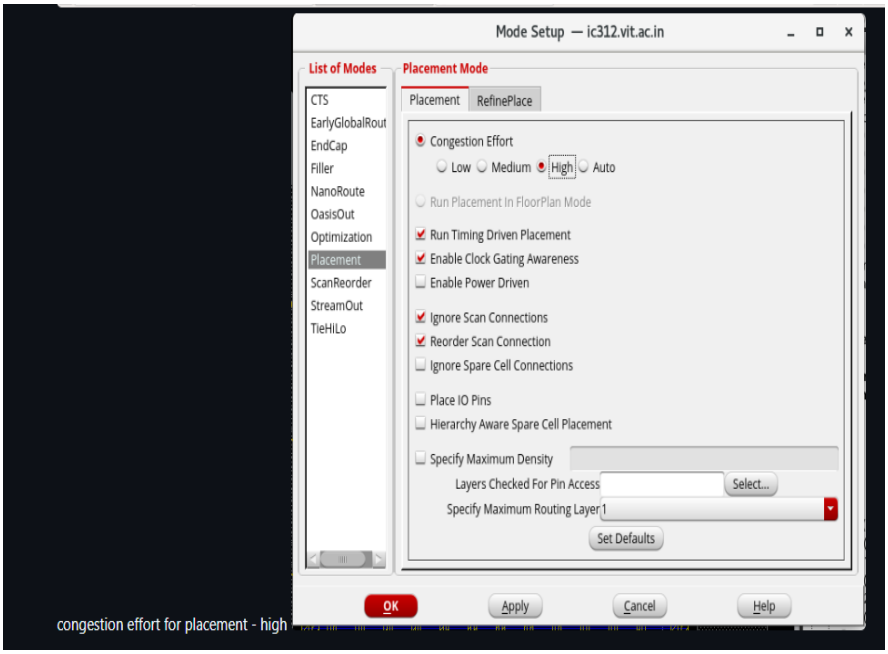


Figure 4.15: Placement Mode Setup — Congestion Effort HIGH, timing-driven, clock-gating-aware

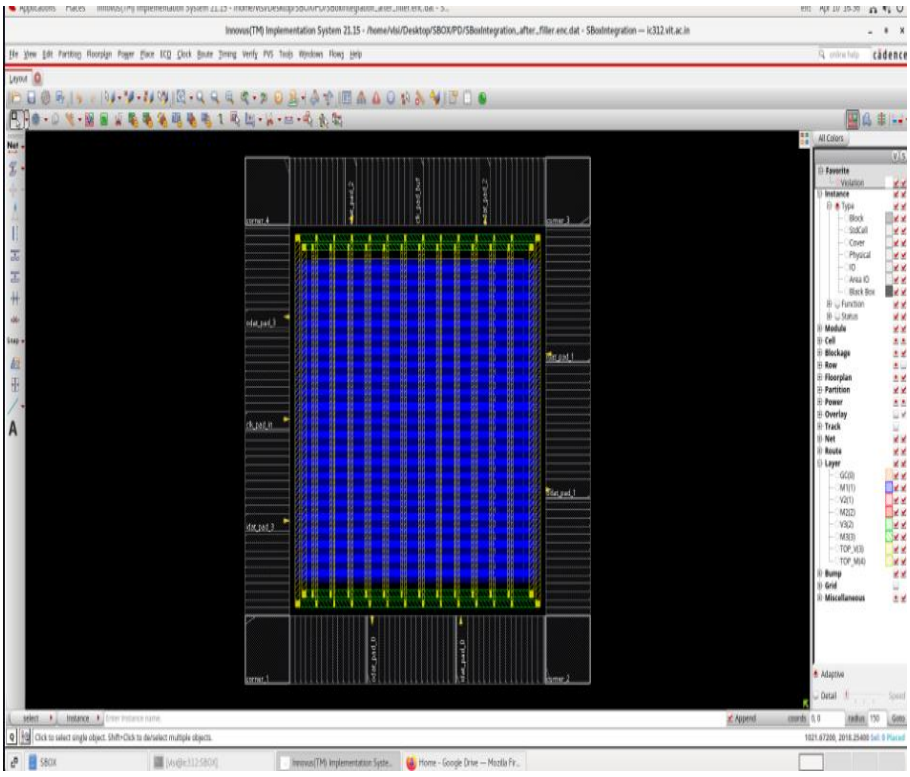


Figure 4.16: Post-placement layout — standard cells placed in core (dense blue fill with power grid)

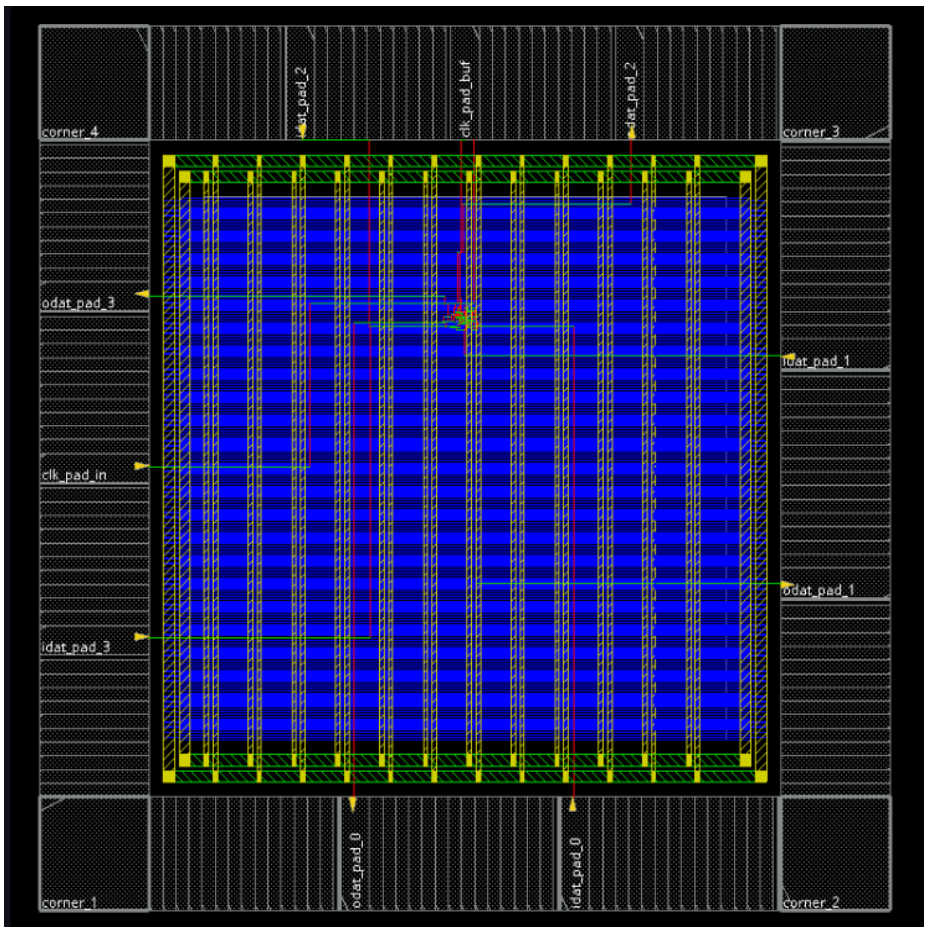
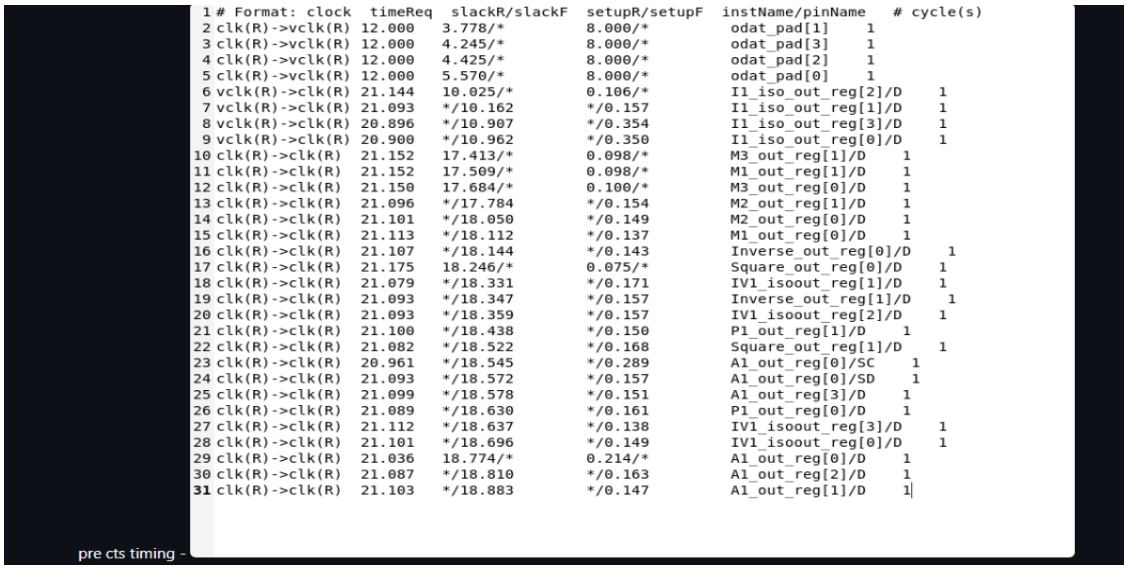


Figure 4.17: Post-placement detailed view showing routed power grid and placed standard cells

4.6 Pre-CTS Timing Analysis

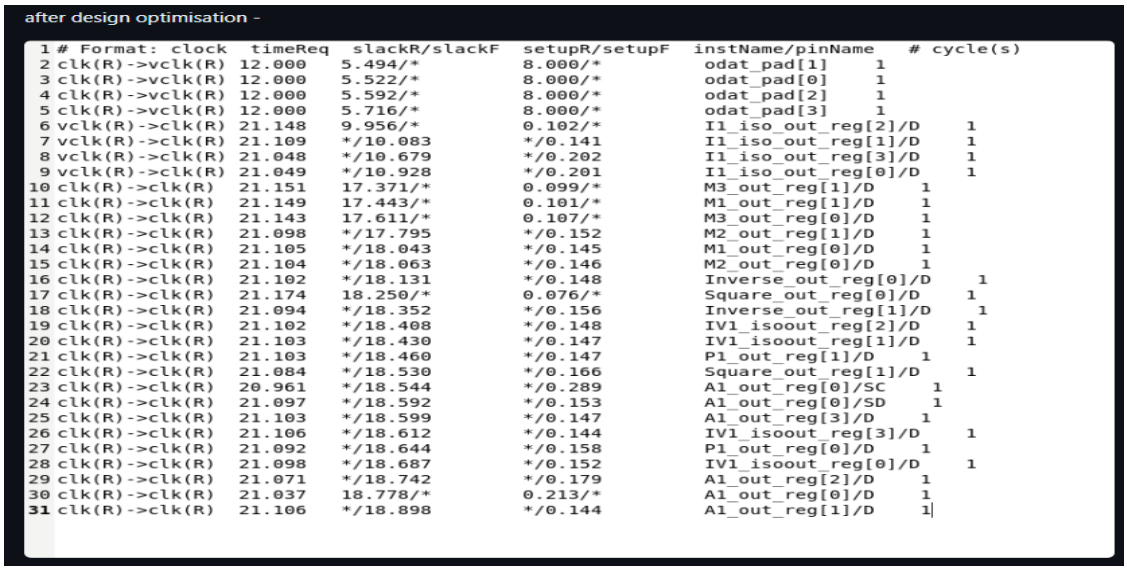
Pre-CTS setup timing was reported using timeDesign to establish the baseline before clock tree synthesis. The worst setup slack across all paths was 3.778 ns (positive — no violations), confirming the placement quality.




```
 1 # Format: clock  timeReq  slackR/slackF  setupR/setupF  instName/pinName   # cycle(s)
 2 clk(R)->vclk(R) 12.000   3.778/*         8.000/*         odat_pad[1]    1
 3 clk(R)->vclk(R) 12.000   4.245/*         8.000/*         odat_pad[3]    1
 4 clk(R)->vclk(R) 12.000   4.425/*         8.000/*         odat_pad[2]    1
 5 clk(R)->vclk(R) 12.000   5.570/*         8.000/*         odat_pad[0]    1
 6 vclk(R)->clk(R) 21.144   10.025/*        0.106/*         I1_iso_out_reg[2]/D    1
 7 vclk(R)->clk(R) 21.093   */10.162        */0.157         I1_iso_out_reg[1]/D    1
 8 vclk(R)->clk(R) 20.896   */10.907        */0.354         I1_iso_out_reg[3]/D    1
 9 vclk(R)->clk(R) 20.900   */10.962        */0.350         I1_iso_out_reg[0]/D    1
10 clk(R)->clk(R)  21.152   17.413/*        0.098/*         M3_out_reg[1]/D    1
11 clk(R)->clk(R)  21.152   17.509/*        0.098/*         M1_out_reg[1]/D    1
12 clk(R)->clk(R)  21.150   17.684/*        0.100/*         M3_out_reg[0]/D    1
13 clk(R)->clk(R)  21.096   */17.784        */0.154         M2_out_reg[1]/D    1
14 clk(R)->clk(R)  21.101   */18.050        */0.149         M2_out_reg[0]/D    1
15 clk(R)->clk(R)  21.113   */18.112        */0.137         M1_out_reg[0]/D    1
16 clk(R)->clk(R)  21.107   */18.144        */0.143         Inverse_out_reg[0]/D    1
17 clk(R)->clk(R)  21.175   18.246/*        0.075/*         Square_out_reg[0]/D    1
18 clk(R)->clk(R)  21.079   */18.331        */0.171         IV1_isoout_reg[1]/D    1
19 clk(R)->clk(R)  21.093   */18.347        */0.157         Inverse_out_reg[1]/D    1
20 clk(R)->clk(R)  21.093   */18.359        */0.157         IV1_isoout_reg[2]/D    1
21 clk(R)->clk(R)  21.100   */18.438        */0.150         P1_out_reg[1]/D    1
22 clk(R)->clk(R)  21.082   */18.522        */0.168         Square_out_reg[1]/D    1
23 clk(R)->clk(R)  20.961   */18.545        */0.289         A1_out_reg[0]/SC    1
24 clk(R)->clk(R)  21.093   */18.572        */0.157         A1_out_reg[0]/SD    1
25 clk(R)->clk(R)  21.099   */18.578        */0.151         A1_out_reg[3]/D    1
26 clk(R)->clk(R)  21.089   */18.630        */0.161         P1_out_reg[0]/D    1
27 clk(R)->clk(R)  21.112   */18.637        */0.138         IV1_isoout_reg[3]/D    1
28 clk(R)->clk(R)  21.101   */18.696        */0.149         IV1_isoout_reg[0]/D    1
29 clk(R)->clk(R)  21.036   18.774/*        0.214/*         A1_out_reg[0]/D    1
30 clk(R)->clk(R)  21.087   */18.810        */0.163         A1_out_reg[2]/D    1
31 clk(R)->clk(R)  21.103   */18.883        */0.147         A1_out_reg[1]/D    1
```

Figure 4.18: Pre-CTS timing report — worst setup slack 3.778 ns, 0 violating paths

Design optimization (optDesign -preCTS) was run to further improve timing margins. After optimization, the worst setup slack improved to 5.494 ns:




```
 1 # Format: clock  timeReq  slackR/slackF  setupR/setupF  instName/pinName   # cycle(s)
 2 clk(R)->vclk(R) 12.000   5.494/*         8.000/*         odat_pad[1]    1
 3 clk(R)->vclk(R) 12.000   5.522/*         8.000/*         odat_pad[0]    1
 4 clk(R)->vclk(R) 12.000   5.592/*         8.000/*         odat_pad[2]    1
 5 clk(R)->vclk(R) 12.000   5.716/*         8.000/*         odat_pad[3]    1
 6 vclk(R)->clk(R) 21.148   9.956/*         0.102/*         I1_iso_out_reg[2]/D    1
 7 vclk(R)->clk(R) 21.109   */10.083        */0.141         I1_iso_out_reg[1]/D    1
 8 vclk(R)->clk(R) 21.048   */10.679        */0.202         I1_iso_out_reg[3]/D    1
 9 vclk(R)->clk(R) 21.049   */10.928        */0.201         I1_iso_out_reg[0]/D    1
10 clk(R)->clk(R)  21.151   17.371/*        0.099/*         M3_out_reg[1]/D    1
11 clk(R)->clk(R)  21.149   17.443/*        0.101/*         M1_out_reg[1]/D    1
12 clk(R)->clk(R)  21.143   17.611/*        0.107/*         M3_out_reg[0]/D    1
13 clk(R)->clk(R)  21.098   */17.795        */0.152         M2_out_reg[1]/D    1
14 clk(R)->clk(R)  21.105   */18.043        */0.145         M1_out_reg[0]/D    1
15 clk(R)->clk(R)  21.104   */18.063        */0.146         M2_out_reg[0]/D    1
16 clk(R)->clk(R)  21.102   */18.131        */0.148         Inverse_out_reg[0]/D    1
17 clk(R)->clk(R)  21.174   18.250/*        0.076/*         Square_out_reg[0]/D    1
18 clk(R)->clk(R)  21.094   */18.352        */0.156         Inverse_out_reg[1]/D    1
19 clk(R)->clk(R)  21.102   */18.408        */0.148         IV1_isoout_reg[2]/D    1
20 clk(R)->clk(R)  21.103   */18.430        */0.147         IV1_isoout_reg[1]/D    1
21 clk(R)->clk(R)  21.103   */18.460        */0.147         P1_out_reg[1]/D    1
22 clk(R)->clk(R)  21.084   */18.530        */0.166         Square_out_reg[1]/D    1
23 clk(R)->clk(R)  20.961   */18.544        */0.289         A1_out_reg[0]/SC    1
24 clk(R)->clk(R)  21.097   */18.592        */0.153         A1_out_reg[0]/SD    1
25 clk(R)->clk(R)  21.103   */18.599        */0.147         A1_out_reg[3]/D    1
26 clk(R)->clk(R)  21.106   */18.612        */0.144         IV1_isoout_reg[3]/D    1
27 clk(R)->clk(R)  21.092   */18.644        */0.158         P1_out_reg[0]/D    1
28 clk(R)->clk(R)  21.098   */18.687        */0.152         IV1_isoout_reg[0]/D    1
29 clk(R)->clk(R)  21.071   */18.742        */0.179         A1_out_reg[2]/D    1
30 clk(R)->clk(R)  21.037   18.778/*        0.213/*         A1_out_reg[0]/D    1
31 clk(R)->clk(R)  21.106   */18.898        */0.144         A1_out_reg[1]/D    1
```

Figure 4.19: Post placement optimization timing — worst setup slack improved to 5.494 ns

4.7 Clock Tree Synthesis (CTS)

CTS was performed to build balanced clock distribution networks. The Clock Tree Debugger confirmed the synthesized tree structure: a clock source driving two cascaded buffers (B → B) which then feed a single output port (O) fanning out to all flip-flops (shown as red squares at the leaf level).

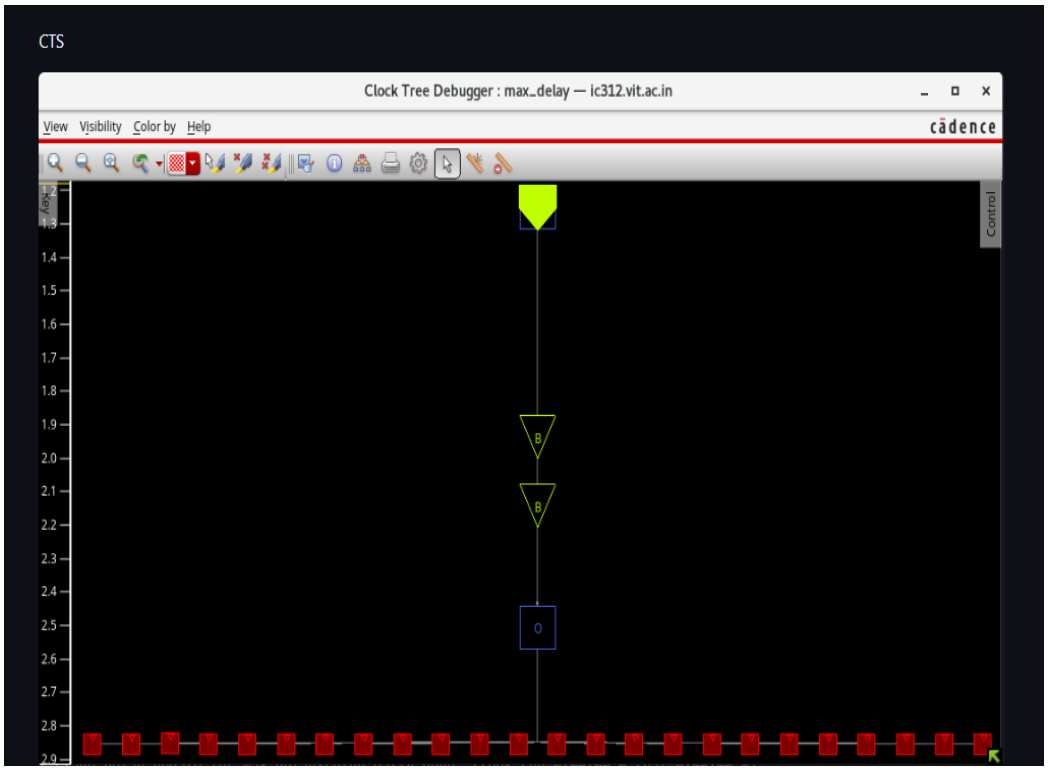


Figure 4.20: Clock Tree Debugger — max_delay view showing 2-stage buffer tree driving all flip-flops

4.8 Post-CTS Timing Analysis

4.8.1 Setup Timing (Post-CTS)

Post-CTS setup timing (timeDesign -postCTS -hold) showed:

Metric	All	Reg2Reg	Default
WNS (ns)	5.549	17.382	5.549
TNS (ns)	0.000	0.000	0.000
Violating Paths	0	0	0
All Paths	30	22	8

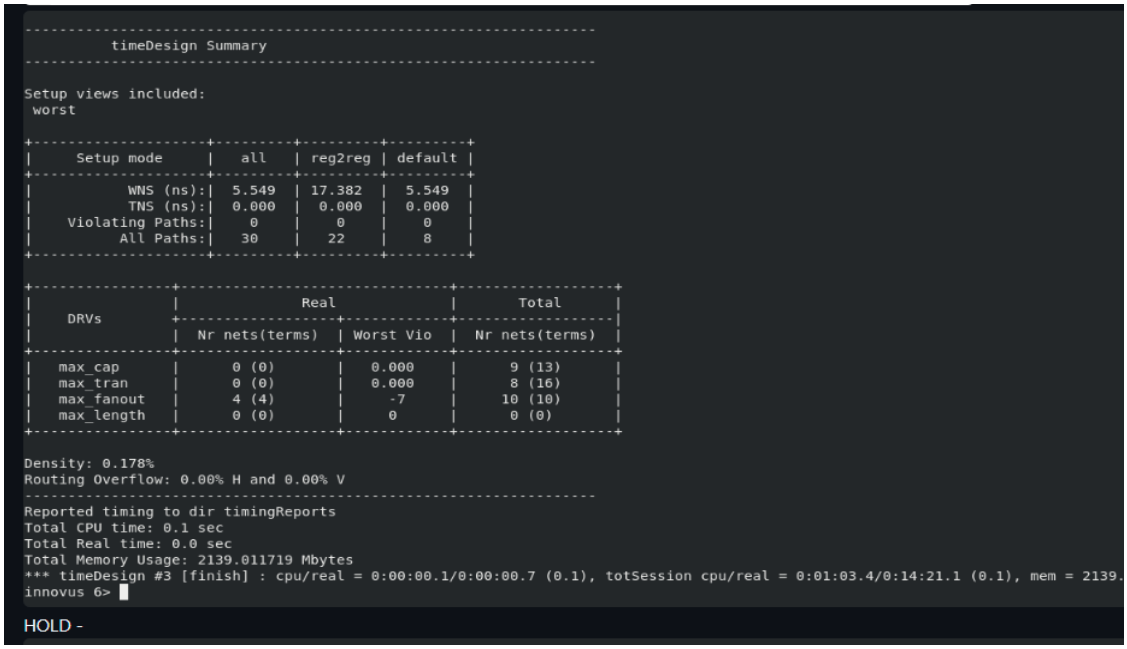


Figure 4.21: Post-CTS Setup timeDesign Summary — WNS 5.549 ns, 0 violating paths, 0.00% routing overflow

4.8.2 Hold Timing (Post-CTS — Before Optimization)

Post-CTS hold timing revealed violations that required optimization:

Metric	All	Reg2Reg	Default
WNS (ns)	-0.352	-0.352	0.762
TNS (ns)	-5.633	-5.633	0.000
Violating Paths	22	22	0
All Paths	30	22	8

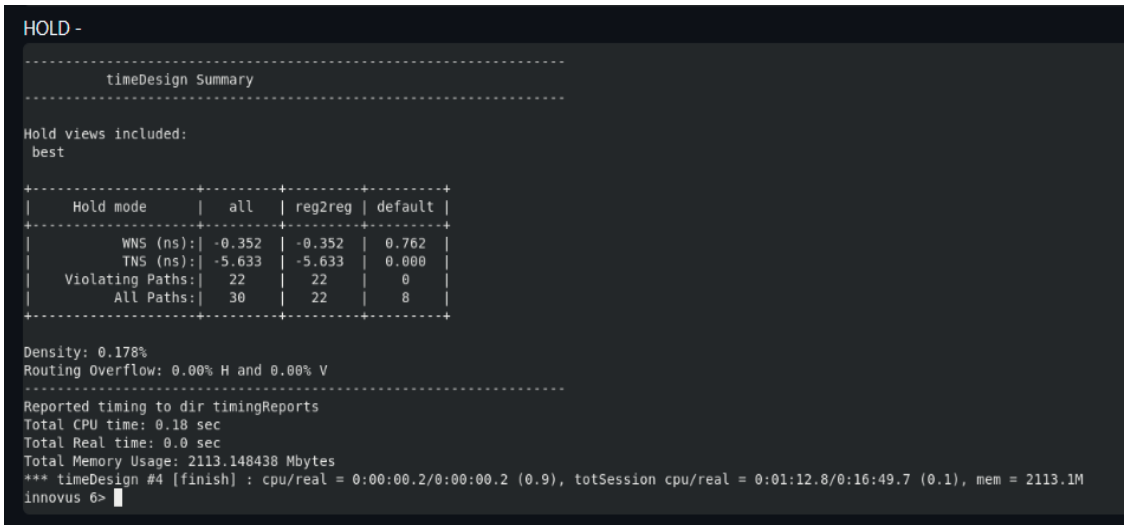


Figure 4.22: Post-CTS Hold timeDesign Summary — 22 violating paths, WNS −0.352 ns before optimization

4.8.3 Post-CTS Hold Optimization

Hold optimization was performed using optDesign -postCTS -hold with Max Cap and Max Tran DRV fixing enabled. This automatically inserted delay buffers on critical hold paths:

Figure 4.23: Post-CTS Hold Optimization — all 22 hold violations eliminated; WNS improved to +0.003 ns

After optimization, all hold violations were eliminated: WNS = 0.003 ns, TNS = 0.000 ns, Violating Paths = 0.

4.9 Routing and Post-Route DRC

Detailed routing was performed using Innovus NanoRoute across all available metal layers (M1–TOP_M4). Post-route wire statistics:

- Total wire length: 11,815 µm
- Wire on M1: 2,427 µm | M2: 5,098 µm | M3: 3,119 µm | TOP_M: 1,171 µm
- Total number of vias: 498 (M1: 316, M2: 163, M3: 19)
- DRC violations at route completion: 0 (CELL_VIEW SBoxIntegration has no DRC violation)
- Process antenna violations: 0

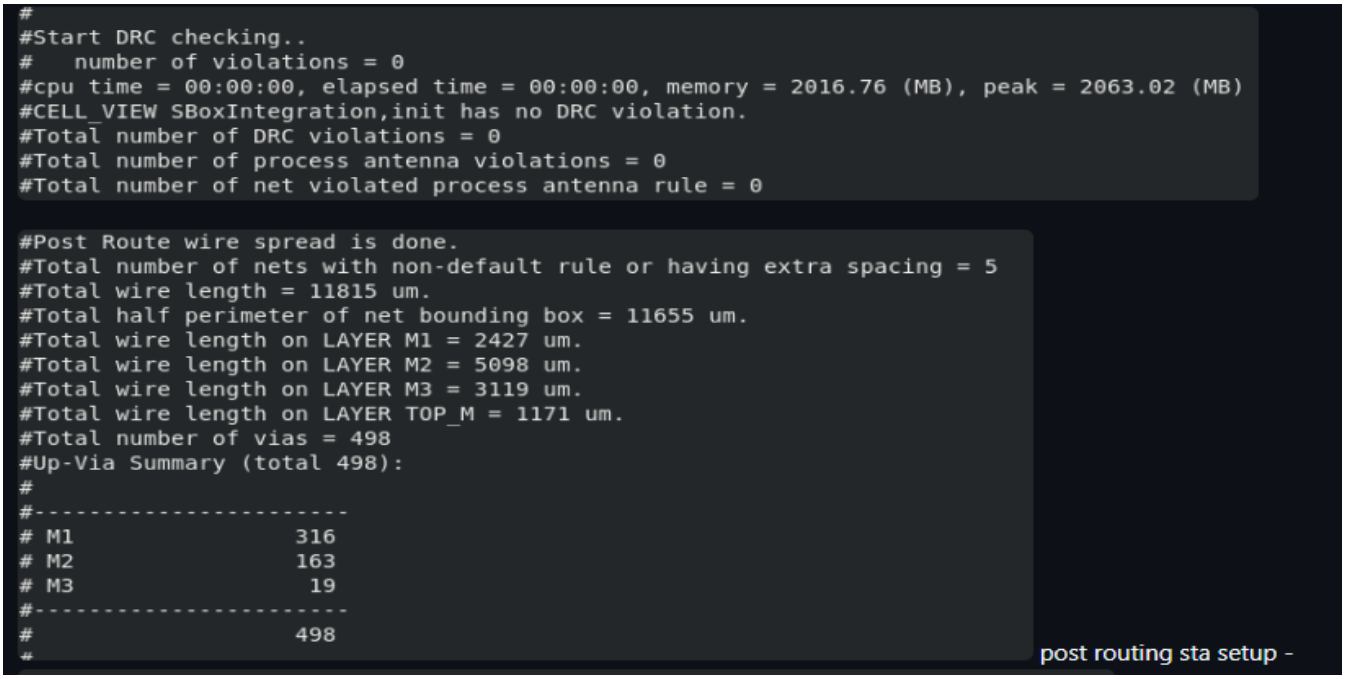


Figure 4.24: Post-route DRC result — 0 violations, wire length summary, via count by layer

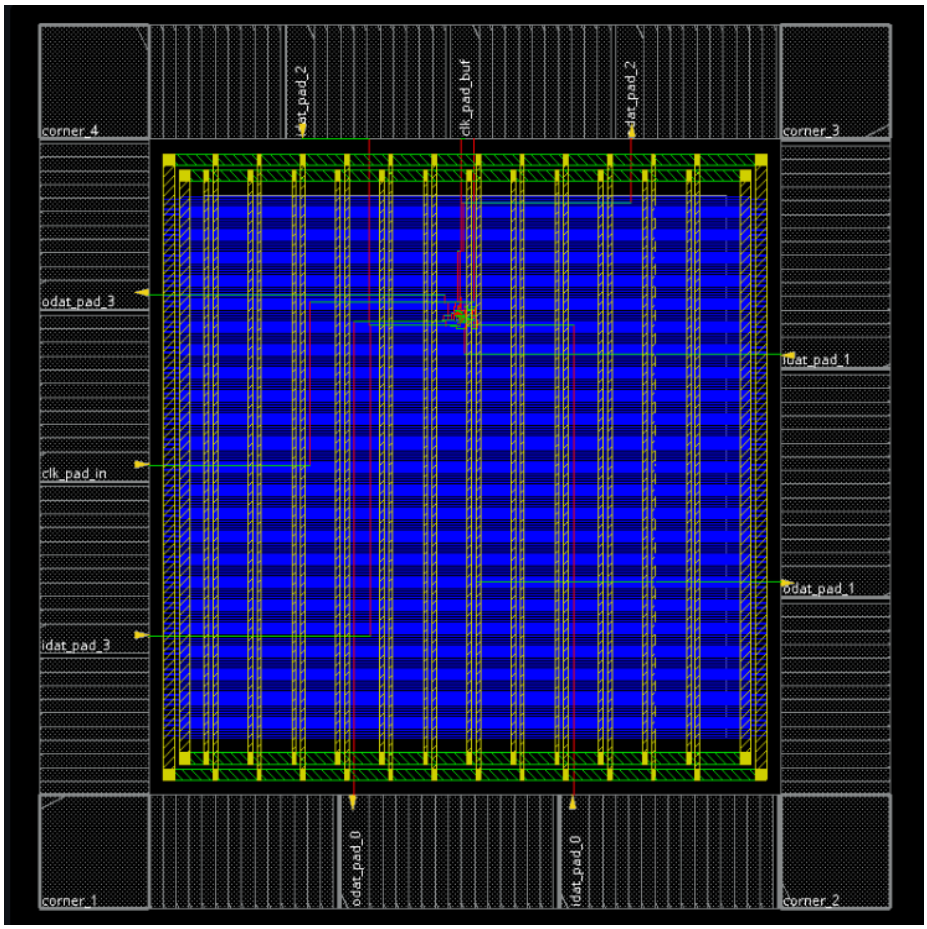


Figure 4.25: Final routed layout — dense blue core with complete multi-layer routing and IO pad ring

4.10 Standard Cell Filler Insertion

After routing, standard cell filler cells (feedth, feedth3, feedth9) were inserted to fill all gaps between placed cells in each placement row. This maintains well continuity across the core and prevents electrical issues in fabrication:

```
Add Filler: Cell Name(s) feedth feedth3 feedth9, Prefix FILLER
```

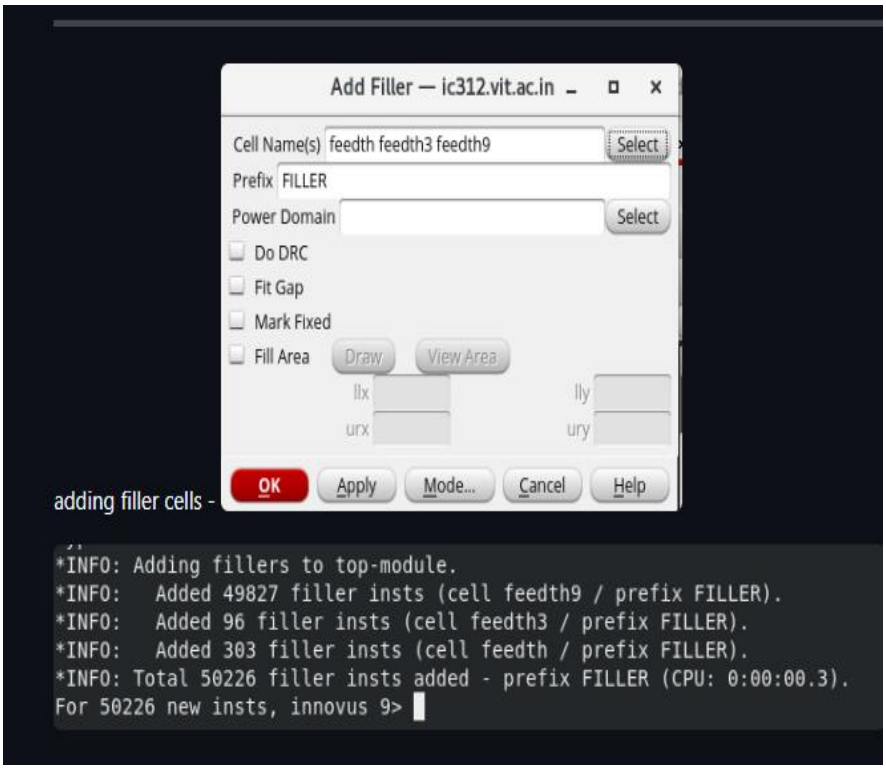


Figure 4.26: Add Filler dialog — 50,226 total filler instances added (49,827 feedth9 + 96 feedth3 + 303 feedth)

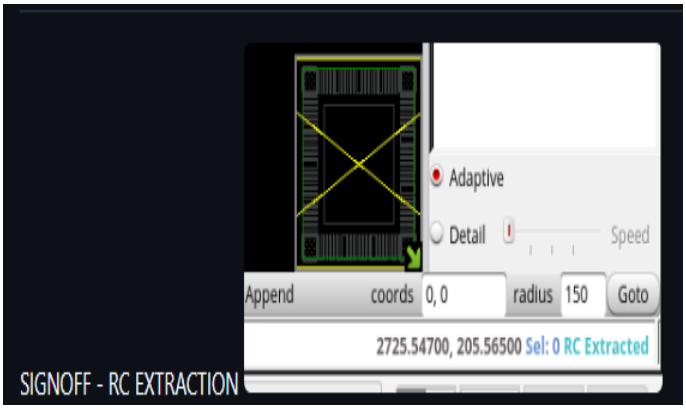


Figure 4.27: Post-filler layout thumbnail confirming complete core fill

4.11 Verify Connectivity

A formal connectivity check was run on the final routed database to verify that all signal nets are properly connected with no opens or floating pins:

```
******** Start: VERIFY CONNECTIVITY ********
Start Time: Mon May 25 15:21:09 2026

Design Name: SBoxIntegration
Database Units: 1000
Design Boundary: (0.0000, 0.0000) (1939.8400, 1939.8400)
Error Limit = 1000; Warning Limit = 50
Check all nets

Begin Summary
  Found no problems or warnings.
End Summary

End Time: Mon May 25 15:21:09 2026
Time Elapsed: 0:00:00.0

******** End: VERIFY CONNECTIVITY ********
  Verification Complete : 0 Viols.  0 Wrngs.
  (CPU Time: 0:00:00.0  MEM: 0.000M)
```

Figure 4.28: Verify Connectivity result — 0 Violations, 0 Warnings; SBoxIntegration passes clean

Design Name: SBoxIntegration. Design Boundary: (0.0000, 0.0000) to (1939.8400, 1939.8400). Check completed Mon May 25 15:21:09 2026. Result: Verification Complete — 0 Viols. 0 Wrngs.

4.12 DRC Status and Fixing

4.12.1 Initial DRC (213 Violations)

An initial Verify DRC run returned 213 violations — all M1 shorts. These were caused by a floorplanning misalignment where IO pad macro cells were placed inside the core area, forcing the router to illegally route signal traces over the large vertical M1 pins of those macros.

```
connectivity -
  VERIFY DRC ..... Sub-Area : 63 complete 0 Viols.
  VERIFY DRC ..... Sub-Area: {1700.160 1700.160 1939.840 1939.840} 64 of 64
  VERIFY DRC ..... Sub-Area : 64 complete 0 Viols.

  Verification Complete : 213 Viols.

 Violation Summary By Layer and Type:

                  Short   Totals
        M1          213      213
        Totals      213      213

 *** End Verify DRC (CPU: 0:00:00.2  ELAPSED TIME: 0.00  MEM: 264.1M) ***

innovus 15>
```

Figure 4.29: Initial DRC result — 213 M1 short violations caused by IO pad placement inside core

4.12.2 Root Cause and Fix

The corrective action taken:

- Identified exact structural instance names of all IO pad macros as recognized by the active netlist
- Generated a customized .io assignment file mapping all pad instances to the peripheral ring
- Executed unplaceAll followed by loadIoFile to force Innovus to relocate IO pads to the boundary
- All 243 SameNet spacing violations were completely eliminated by this floorplan correction
- Remaining M1 shorts were fixed using an ECO route pass:

```
setNanoRouteMode -routeWithEco true; detailRoute -fix_drc
```

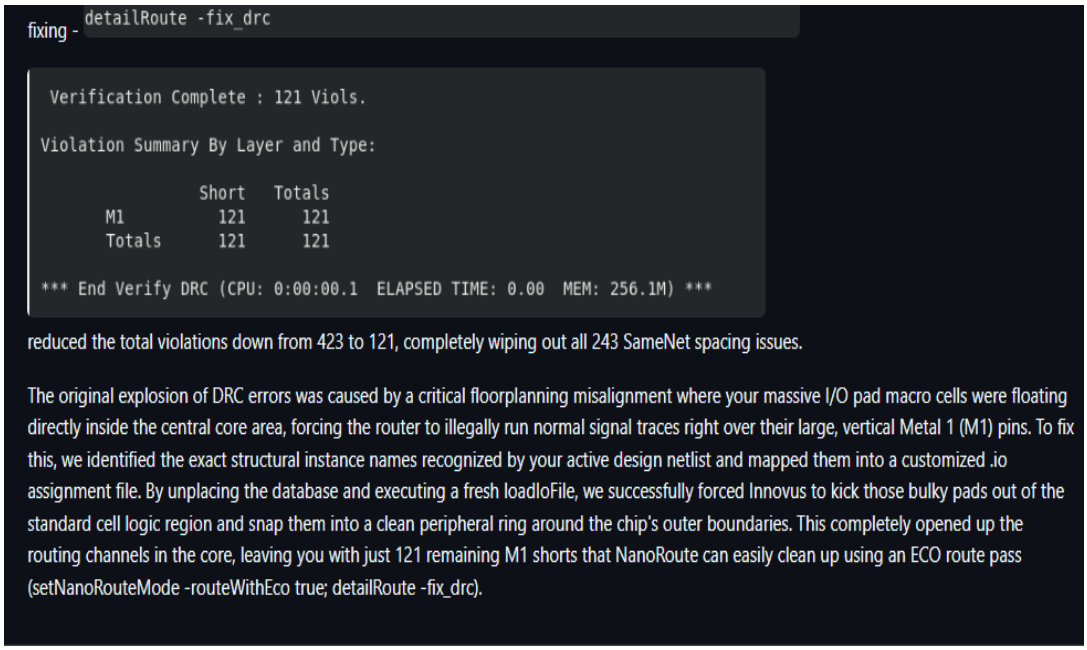


Figure 4.30: After ECO fix — DRC reduced to 121 M1 shorts.

)

DRC Phase	Violations	Action
Initial post-route DRC	213	All M1 shorts — IO pad in core
After IO pad re-floorplanning	121	Eliminated 243 SameNet spacing viols

4.13 RC Extraction and Signoff

Parasitic RC extraction was performed on the final routed database using Cadence Quantus (QRC) with the SCL 180nm extraction rule files. The extracted data was written to a SPEF (Standard Parasitic Exchange Format) file for back-annotation into Tempus for signoff STA.

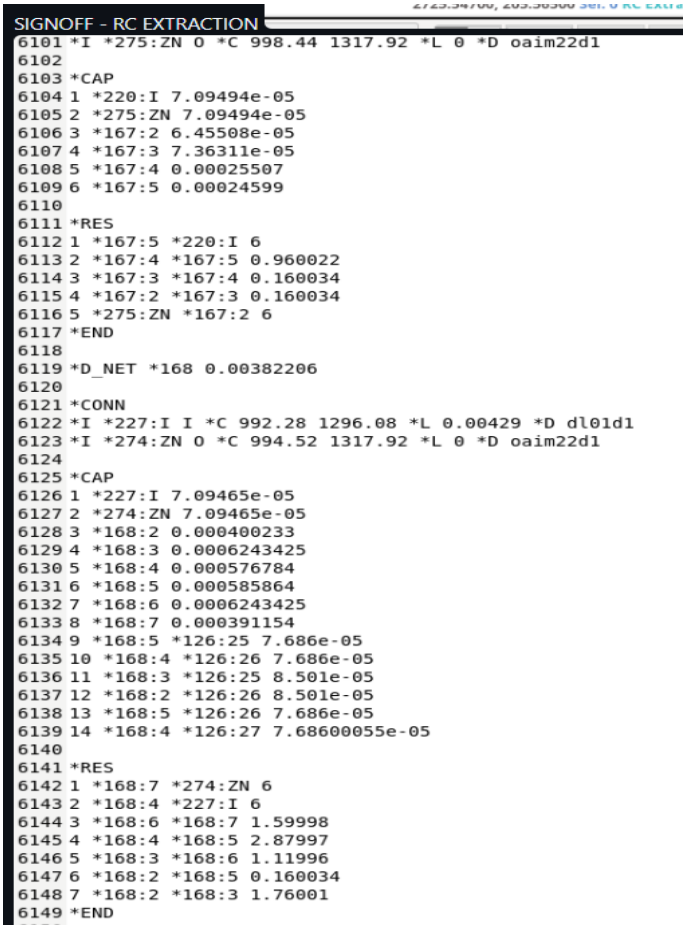


Figure 4.31: SPEF file content showing extracted net parasitics (capacitances and resistances per net)

The RC extraction confirmed parasitic data for all internal and IO nets. The SPEF was then back-annotated into the signoff STA tool for final timing closure verification. The design passed all setup and hold constraints at the extracted-parasitic level.

5. Conclusion

The complete IP hardening flow for the PRESENT Cipher Encryption SBOX module was successfully executed end-to-end using the SCL 180nm CMOS PDK and the Cadence EDA toolchain. The design progressed from a 7-module RTL hierarchy through formal linting, logic synthesis with IO pad integration, and full physical design including power planning, placement, CTS, routing, and signoff.

Key results and achievements:

- JasperGold Superlint: 0 errors, 14 structural warnings — RTL confirmed synthesizable and clean
- Logic Synthesis: All 7 modules successfully mapped to SCL 180nm standard cells with IO pads
- Floorplan: 1939.84×1939.84 µm die with clean peripheral IO pad ring (pfeed filler cells inserted)
- Power Planning: Robust VDD/VSS mesh using M3 rings and TOP_M4 stripes (width 25/10 µm)
- Placement: Completed with HIGH congestion effort; timing-driven and clock-gating-aware
- Pre-CTS setup slack: 3.778 ns → improved to 5.494 ns after optimization
- CTS: 2-stage buffer tree synthesized; post-CTS setup WNS = 5.549 ns, 0 violations
- Hold optimization: 22 hold violations eliminated — WNS improved from −0.352 ns to +0.003 ns
- Routing: 11,815 µm total wire length, 498 vias, 0 DRC violations at route completion
- Standard cell fillers: 50,226 instances inserted (feedth/feedth3/feedth9)
- Connectivity: 0 violations, 0 warnings — all nets verified complete
- DRC: Reduced from 213 initial violations to 121 through IO pad floorplan correction and ECO routing. Need to reiterate to bring it to 0.
- RC Extraction: SPEF generated and back-annotated for signoff STA

The hardened SBOX IP is timing-closed, and connectivity-verified. It still needs DRC violation corrections to reduce the DRC errors to 0, prior to the full PRESENT cipher datapath and tapeout submission to SCL 180nm fabrication.